

Section 1. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier search process, accepting patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the dictionary against actual counts.

Section 2. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the dictionary against actual counts.

Section 3. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier search process, accepting patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the dictionary against actual counts.

Section 4. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier search process, accepting patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the dictionary against actual counts.

Section 5. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the dictionary against actual counts.

Section 6. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier search process, accepting patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the dictionary against actual counts.

Section 7. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern
Section 8. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patte
Section 9. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns
Section 10. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when th
Section 11. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 12. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionar

Section 13. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and paying for itself.

Section 14. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and paying for itself.

Section 15. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and paying for itself.

Section 16. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and paying for itself.

Section 17. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and paying for itself.

Section 18. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and paying for itself.

Section 19. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 20. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 21. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier s
Section 22. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thre
Section 23. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision En
Section 24. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adapti

Section 25. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol blocks.

Section 26. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol blocks.

Section 27. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol blocks.

Section 28. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol blocks.

Section 29. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol blocks.

Section 30. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol blocks.

Section 31. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts

Section 32. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 33. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 34. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 35. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 36. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself

Section 37. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman
Section 38. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed s
Section 39. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pat
Section 40. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision En
Section 41. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural block
Section 42. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream wi

Section 43. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 44. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of fr
Section 45. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks,
Section 46. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, a
Section 47. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier s
Section 48. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted pattern

Section 49. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the results
Section 50. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical
Section 51. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 52. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays
Section 53. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual
Section 54. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against a

Section 55. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of fr
Section 56. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost
Section 57. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed s
Section 58. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of fr
Section 59. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream wi
Section 60. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and codin

Section 61. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using a Bit Cost Decision Engine

Section 62. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 63. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using a Bit Cost Decision Engine

Section 64. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 65. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 66. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 67. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tiered approach.
 Section 68. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.
 Section 69. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.
 Section 70. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tiered approach, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acceptance until the desired compression level is reached.
 Section 71. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acceptance until the desired compression level is reached.
 Section 72. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis.

Section 73. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.
Section 74. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger patterns.
Section 75. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary.
Section 76. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses the dictionary.
Section 77. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses the dictionary.
Section 78. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses the dictionary.

Section 79. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost
Section 80. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 81. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only w
Section 82. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pat
Section 83. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier s
Section 84. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resu

Section 85. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine
Section 86. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File C
Section 87. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting e
Section 88. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 89. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier s
Section 90. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dic

Section 91. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed structure

Section 92. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed structure

Section 93. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed structure

Section 94. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed structure

Section 95. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed structure

Section 96. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed structure

Section 97. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns

Section 98. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns

Section 99. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through

Section 100. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and

Section 101. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of

Section 102. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of

Section 103. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 104. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr
Section 105. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 106. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 107. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary agains
Section 108. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d

Section 109. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself
Section 110. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds
Section 111. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 112. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual code
Section 113. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 114. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the

Section 115. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual data.

Section 116. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 117. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thresholding process.

Section 118. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 119. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 120. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.

Section 121. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 122. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one pass.

Section 123. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing each block.

Section 124. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing each block.

Section 125. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 126. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 127. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 128. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 129. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical
Section 130. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E
Section 131. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 132. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed

Section 133. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the
Section 134. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 135. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 136. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 137. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hy
Section 138. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa

Section 139. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 140. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 141. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 142. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adap
Section 143. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 144. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p

Section 145. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 146. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream

Section 147. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a

Section 148. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 149. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern

Section 150. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself

Section 151. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 152. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse

Section 153. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only

Section 154. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only

Section 155. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p

Section 156. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte

Section 157. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual co
Section 158. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 159. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 160. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 161. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pa
Section 162. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr

Section 163. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 164. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural patterns

Section 165. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural patterns

Section 166. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself

Section 167. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 168. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural patterns

Section 169. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 170. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v
Section 171. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 172. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build
Section 173. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 174. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 175. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, counting the occurrences of each pattern, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adapting the Huffman tree to the mixed symbol stream.

Section 176. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adapting the Huffman tree to the mixed symbol stream.

Section 177. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary.

Section 178. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its cost.

Section 179. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary.

Section 180. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, counting the occurrences of each pattern, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adapting the Huffman tree to the mixed symbol stream.